FACULTY OF INDUSTRIAL AND ENERGY TECHNOLOGY

Information Technology Department

Web Development II

Project Brief

Supervisor	Dr. Ashraf Abd Alaziz
Course	Web Development II
Technology Stack	PHP Native · OOP · SQL
Total Marks	20 + Bonus Points
Options	Option 1 (5 Bonus) | Option 2 (10 Bonus)

Choose ONE option to implement

Project Overview

This project requires you to build a complete, full-stack web application using native PHP following Object-Oriented Programming (OOP) principles, backed by a relational SQL (MySQL) database. You must choose one of the two options below and implement it fully.

The application must include user authentication with role-based access control, a responsive front-end (HTML/CSS), and secure database interactions using Prepared Statements to prevent SQL injection.

Core Technical Requirements (Both Options)

PHP & OOP Architecture

- Use native PHP — no frameworks (no Laravel, no Symfony, no CodeIgniter)
- Organize code using PHP Classes with proper OOP concepts:
  - Encapsulation — keep properties private/protected, expose via methods
  - Inheritance — e.g., Admin, Student, Doctor, Patient all extend a base User class
  - Abstraction — create a base Model class with shared CRUD logic
  - Polymorphism — override methods per role where needed
- Minimum recommended classes: Database, User, Admin, and one class per data entity

Session Management

- Use session_start() at the top of every page
- Store logged-in user ID, name, and role in $_SESSION
- Protect every restricted page by checking the session and redirecting unauthorized users

Security & Validation

- ALL database queries must use Prepared Statements (PDO or MySQLi)
- Never use raw string concatenation in SQL queries
- Validate all form inputs server-side (not just client-side)
- Use password_hash() for storing passwords and password_verify() for login
- Sanitize all output using htmlspecialchars() to prevent XSS

Database Design

- Use MySQL with proper relational design (Primary Keys, Foreign Keys)
- Use INT AUTO_INCREMENT for primary key columns
- Use ENUM or VARCHAR for role columns
- Ensure all table relationships are enforced (ON DELETE CASCADE where appropriate)

Option 1: University Portal System

Total Marks	20 Marks + 5 Bonus
User Roles	Admin, Student
Core Technology	PHP OOP + MySQL + PHP Sessions

System Description

The University Portal is a web-based management system that serves two types of users: administrators who manage university data, and students who browse available academic information. The system centralizes departments, courses, and news in one place with controlled access per role.

Feature Requirements

Feature Area	Requirements
Authentication	Register with name, email, password, and role selection. Login using email and password with session-based access. Logout clears the session. Passwords must be hashed using password_hash().
Admin: Departments	Full CRUD — Create, Read, Update, Delete departments. Each department has a name and description. Validation required on all fields.
Admin: Courses	Full CRUD on courses. Each course must be linked to a department (foreign key). Fields: course name, code, description, department_id.
Admin: News	Create, edit, and delete news/announcements. Fields: title, content, published date.
Student: View Courses	Students can browse all available courses filtered by department. No add/edit/delete access.
Student: View Departments	Students can view a list of all departments and their details.
Student: View News	Students can read all published news and announcements.
Student: Enrollment (Optional)	Students may enroll in courses. An enrollment record links student_id to course_id. Students should not enroll twice in the same course.

Database Schema

Required tables and their key columns:

Table	Columns
users	`id (PK), name, email, password (hashed), role ENUM('admin','student'), created_at`
departments	`id (PK), name, description, created_at`
courses	`id (PK), name, code, description, department_id (FK → departments.id), created_at`
news	`id (PK), title, content, published_at, created_by (FK → users.id)`
enrollments (optional)	`id (PK), student_id (FK → users.id), course_id (FK → courses.id), enrolled_at — UNIQUE(student_id, course_id)`

Suggested OOP Class Structure

- Database — Singleton class managing PDO connection
- User — Base class: id, name, email, password, role; methods: register(), login(), logout()
- Admin extends User — manageUsers(), createDepartment(), updateCourse(), etc.
- Student extends User — viewCourses(), viewDepartments(), enrollInCourse()
- Department — id, name, description; CRUD methods
- Course — id, name, code, department_id; CRUD methods
- News — id, title, content; CRUD methods
- Enrollment (optional) — student_id, course_id; enroll(), getByStudent()

Bonus Features (+5 Marks)

Bonus Feature	Description	Marks
Search Functionality	Search bar to find courses or departments by keyword using SQL LIKE queries	**+2**
Dashboard Statistics	Admin dashboard showing total users, departments, courses, recent enrollments	**+2**
Student Profile Page	Student can view and update their profile (name, email, password)	**+1**

Option 2: National Health Database System

Total Marks	20 Marks + 10 Bonus
User Roles	Admin, Doctor, Patient
Core Technology	PHP OOP + MySQL + Role-Based Access Control

System Description

The National Health Database System is a secure, multi-role medical records platform. Doctors create and manage patient records and prescriptions. Patients can access only their own medical history. Administrators oversee all user accounts. Role-based data access is the central challenge of this option.

Feature Requirements

Feature Area	Requirements
Authentication	Register with name, email, password, and role. Login with email and password using PHP Sessions. Logout clears session. Passwords hashed with password_hash().
Doctor: Medical Records	Add new medical records linked to a specific patient. Update diagnosis on existing records. View all records for their assigned patients.
Doctor: Prescriptions	Add prescriptions to a medical record. Each prescription has medication name, dosage, and instructions.
Patient: Medical History	View ONLY their own medical records — never another patient's. Records show date, doctor name, and diagnosis.
Patient: Prescriptions	View prescriptions attached to their records, including medication and dosage.
Patient: Profile	Update personal data: name, email, and password.
Admin: User Management	View all users. Add new doctors and patients. Delete users. Cannot delete themselves.

Database Schema

Table	Columns
users	`id (PK), name, email, password (hashed), role ENUM('admin','doctor','patient'), phone, created_at`
medical_records	`id (PK), patient_id (FK → users.id), doctor_id (FK → users.id), diagnosis, notes, visit_date, created_at`

Table	Columns
prescriptions	`id (PK), record_id (FK → medical_records.id), medication_name, dosage, instructions, prescribed_at`

Suggested OOP Class Structure

- Database — Singleton PDO connection class
- User — Base class with shared properties and methods: register(), login(), logout(), updateProfile()
- Admin extends User — getUsers(), addDoctor(), addPatient(), deleteUser()
- Doctor extends User — addRecord(), updateDiagnosis(), addPrescription(), getMyPatients()
- Patient extends User — getMyRecords(), getMyPrescriptions()
- MedicalRecord — patient_id, doctor_id, diagnosis, notes; CRUD methods with role checks
- Prescription — record_id, medication, dosage, instructions; add(), getByRecord()

Critical Security Note

Every database query that fetches records must filter by the logged-in user's role. A patient must ONLY see records where patient_id = $_SESSION['user_id']. A doctor must ONLY see records where doctor_id = $_SESSION['user_id']. Failing to enforce this is a critical security flaw and will result in mark deductions.

Bonus Features (+10 Marks)

Bonus Feature	Description	Marks
Patient Search	Doctors and admins can search for patients by name or email using SQL LIKE	**+2**
Dashboard Statistics	Role-aware dashboard: admin sees total counts; doctor sees their patient count and recent records	**+3**
Medical History Tracking	Timeline view of a patient's records sorted by visit date, showing progression of treatment	**+3**
Advanced Role-Based Access	Middleware-style PHP class that checks permissions before any action; prevents unauthorized access programmatically	**+2**

Grading Criteria

Both options are graded on the same 20-mark rubric. Points are awarded across the following categories:

Criteria	Marks
Authentication & Session Management Register, login, logout working with sessions	**4 pts**
Role-Based Access Control Each role sees and can do only what is permitted	**4 pts**
CRUD Operations Complete Create, Read, Update, Delete for all entities	**4 pts**
OOP Code Structure Use of classes, inheritance, encapsulation, proper organization	**3 pts**
Database Design Correct tables, relationships, foreign keys, proper normalization	**3 pts**
Security & Validation Prepared statements, input validation, password hashing	**2 pts**

Bonus marks are awarded on top of the 20-mark score. Option 2 offers up to +10 bonus marks versus Option 1's +5, making it potentially more rewarding for stronger students.

Tip: Option 2's higher bonus ceiling (+10 vs +5) makes it the better choice if you are comfortable implementing multi-role permission logic and relational medical data. Option 1 is cleaner and faster to implement if you want a reliable base score.

Submission Checklist

Before submitting, verify the following:

- PHP files use OOP classes (not procedural style)
- All SQL queries use Prepared Statements — no raw string concatenation
- Passwords are stored hashed using password_hash()
- Login uses password_verify() to authenticate
- Sessions are used to track logged-in user and role
- Every restricted page checks the session and redirects if unauthorized
- All form inputs are validated server-side
- Database has proper foreign key relationships
- Admin cannot access student/patient pages and vice versa
- All CRUD operations work correctly (Create, Read, Update, Delete)
- Code is organized in folders: /classes, /pages or /views, /includes, /config